

Refinamiento iterativo: puliendo resultados del Álgebra Lineal

Prof. Miguel Ángel Norzagaray Cosío
Universidad Autónoma de Baja California Sur
manc@uabcs.mx

3 de noviembre de 2023

Índice

1. Introducción	1
2. La aritmética de la computadora	2
3. Errores en la solución de $Ax = b$	2
4. Refinamiento iterativo	3
5. Uso de mayor precisión	4
6. Conclusiones	5

Resumen

Casi todo se resuelve con computadora, pero, por su precisión limitada, se introducen errores. Se presenta una técnica que permite disminuir los errores, introducida desde los inicios del uso de computadoras.

Palabras clave: error, refinamiento iterativo.

1. Introducción

En Álgebra Lineal (se escribirá AL de ahora en adelante) se dice que no hay aplicación seria que no ocurra con computadora [2]. En el salón de clases se pueden resolver problemas de pocas dimensiones, de 2 a 4. Tamaños mayores son muy propensos a errores y es preferible hacerlo con computadoras.

En la práctica, los problemas suelen involucrar matrices de decenas de ecuaciones y otras tantas variables. A veces miles e incluso decenas de miles. Por ello es necesario disponer de algoritmos eficientes que reduzcan el error cometido por las computadoras. Una de las primeras de estas técnicas fue desarrollada a finales de la década de 1950 por James Wilkinson y presentado en [6].

2. La aritmética de la computadora

A la manera como la computadora hace cálculos numéricos se le llama Aritmética de Punto Flotante (APF). Sus detalles está definidos formalmente en [1] y existe un artículo clásico de Goldberg [3] que trae los detalles importantes.

De manera sintética, los números que usa la computadora se guardan en grupos de 4 u 8 bytes (formatos de simple y doble precisión). Un número real x se codifica en los 32 o 64 bits de cada formato. Se emplea una forma similar a la notación científica

$$x = (-1)^s \times m \times 10^e$$

donde s es indica el signo, la m es la mantisa y e el exponente. Por ejemplo, el número $\pi = 3.141592653589793238 \dots$ se representa como

$$\pi = (-1)^0 \times 3.141592653589793238 \dots \times 10^0$$

y por lo general la mantisa se deja en un rango, como $1 \leq m < 10$.

Es importante aclarar que el ejemplo anterior se muestra en base 10, pero las computadoras trabajan en base 2 o binaria. La aritmética en base dos tiene características especiales que son explotadas eficientemente en la APF.

Como la cantidad de bits de los formatos es finita, los números reales con fracción infinita deben representarse con aproximaciones, obtenidas por redondeo y a veces por truncamiento. Esto significa que hay un error al representar números con fracción infinita como $1/3$, $\sqrt{2}$ o π .

En el formato de doble precisión, se utiliza un bit para el signo, 11 para el exponente y 52 para la mantisa. Más detalles de los formatos pueden consultarse en [4] de esta misma serie de lecturas breves.

3. Errores en la solución de $Ax = b$

Si se usa la aritmética nativa de una computadora para resolver un sistema de ecuaciones lineales, es natural que luego de todas las operaciones redondeadas los errores se hayan acumulado. Por lo mismo, la solución encontrada no es exacta. En el sistema $Ax = b$, se calcula $x = A^{-1}b$ y debiera ocurrir que Ax es igual a b , pero lo que se tiene es

$$Ax = b + e$$

y e es el error cometido por la computadora. Esto significa que realmente se tiene la solución exacta de un sistema parecido de la forma

$$A(x + r) = b + e$$

y r es el error en nuestra solución. En análisis numérico estos números tiene su propio nombre: a r se le llama el error hacia atrás (backward error) y a e error hacia adelante (forward error) y siempre es deseable conocerlos, para todo tipo de problemas, no solo de álgebra lineal.

4. Refinamiento iterativo

El análisis de la sección anterior permite aprovechar los errores para corregir la solución, de la siguiente manera.

1. Se resuelve $Ax = b$.
2. Se calcula $e = b - Ax$.
3. Se resuelve $Ar = e$.
4. La nueva solución es $x + r$.

Se espera que la nueva solución se más exacta que la primera en el sentido de que $A(x + r)$ es más parecida a b que Ax , lo que puede medirse con

$$\|b - Ax\| < \|b - A(x + r)\|$$

para comprobar.

Veámoslo con un ejemplo. Sea el sistema $Ax = b$ siguiente

$$\begin{bmatrix} -3 & 7 & -1 & -8 & -1 \\ 1 & 4 & 4 & 5 & 1 \\ -2 & -5 & -2 & 5 & -5 \\ 2 & 3 & -3 & 8 & 2 \\ 3 & -1 & 4 & 0 & -2 \end{bmatrix} x = \begin{bmatrix} 37 \\ 2 \\ -25 \\ -7 \\ -3 \end{bmatrix}$$

donde la matriz A se ha generado aleatoriamente y el vector b es la combinación lineal de 3 veces la segunda columna de A menos 2 veces la cuarta columna. La solución que se espera es $x = (0, 3, 0, -2, 0)$, pero resolviendo de la forma abitual en Matlab u Octave con el comando $\mathbf{x}=\mathbf{A} \backslash \mathbf{b}$, se obtiene el vector

$$x = \begin{bmatrix} 2.368475785867000 \times 10^{-15} \\ 3.0000000000000001 \\ 7.520342823738286 \times 10^{-16} \\ -2 \\ 6.014503338752520 \times 10^{-16} \end{bmatrix}$$

y es claro que hay errores numéricos. Si ahora multiplicamos la matriz A por x , en vez de obtener el vector b se obtiene otro muy cercano, lo que se evidencia si calculamos $b - Ax$:

$$b - Ax = \begin{bmatrix} 0 \\ -1.021405182655144 \times 10^{-14} \\ 1.421085471520200 \times 10^{-14} \\ -8.881784197001252 \times 10^{-15} \\ -7.993605777301127 \times 10^{-15} \end{bmatrix}$$

y los errores son claros. Analicemos esto con más detalle.

El error $e = b - Ax$, es el producto de las operaciones imprecisas de la computadora. El vector solución x ha sido perturbado por alguna cantidad r , así que no hemos conseguido la solución del sistema, sino una muy cercana denotada por $x + r$. Al multiplicarla por A obtenemos $A(x + r)$ y distribuyendo se llega a

$$Ax + Ar = b + e$$

y como $Ax = b$, cancelamos a ambos lados de la igualdad y obtenemos

$$Ar = e$$

Este sistema no cuesta mucho resolverlo pues ya tenemos A^{-1} pues se calculó al resolver $Ax = b$ de manera inicial. De esta forma, calculamos $r = A^{-1}e$ y bastará actualizar el valor de x con $x + r$. La mejora puede notarse comparando la norma de los errores:

$$\begin{aligned} \|b - Ax\| &= 2.119101390441385 \times 10^{-14} \\ \|b - A(x + r)\| &= 8.018239443638015 \times 10^{-15} \end{aligned}$$

y la ganancia no es mayor sólo porque se sigue usando la misma precisión inicial para calcular los errores y la última aproximación $x + r$. Este refinamiento de la solución inicial puede realizarse una o dos veces y ya no habrá mejora, a menos que se emplee aritmética más precisa en los cálculos posteriores a $x = A^{-1}b$.

Cabe aclarar que no se requiere calcular la inversa de A , lo mejor es factorizarla usando LU o QR y emplear la misma factorización en la solución de $Ar = e$.

5. Uso de mayor precisión

Se puede hacer aplicando operaciones aumentadas [5], que aparecen en la norma de 754 de IEEE de 2018. No se recomienda el uso de aritmética simbólica debido a su alta complejidad temporal. Las rutinas se tardan mucho más en ser ejecutadas.

6. Conclusiones

El refinamiento iterativo es un procedimiento que puede reducir el error numérico cometido al resolver un sistema de ecuaciones lineales con computadora usando su aritmética nativa. En la práctica, pocos pasos llegan al límite de la precisión de una computadora. Su uso está recomendado cuando se requiere la mayor precisión posible.

Referencias

- [1] COMMITTEE, M. S. IEEE standard for binary floating-point arithmetic. *ANSI/IEEE Std 754-1985* (Oct 1985), 1–20. 2
- [2] COWEN, C. C. Centrality of Linear Algebra, 1997. 1
- [3] GOLDBERG, D. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys* 23, 1 (Marzo 1991), 5–48. 2
- [4] NORZAGARAY-COSÍO, M. A. Números de Punto Flotante. Lecturas para estudiantes, May 2023. 2
- [5] RIEDY, J., AND DEMMEL, J. Augmented arithmetic operations proposed for ieee-754 2018. In *2018 IEEE 25th Symposium on Computer Arithmetic (ARITH)* (June 2018), pp. 45–52. 4
- [6] WILKINSON, J. H. *Rounding Errors in Algebraic Processes*. Prentice-Hall, Englewood Cliffs, New Jersey, 1964. 2